Peekaboo: Improving Deployed Decision Models from Right / Wrong Feedback Alone

Abhijith Neil Abraham Fariz Rahman Fadil Rahman

October 2026

Abstract

Vega and Laya are small models that answer typed questions (choose one of these options, yes or no, a rating from one to five) and return a probability for every option in a few milliseconds. Each ships with a recipe for getting better at a user's own task, and in both cases the recipe wants labelled rows. A deployed model almost never has those. What it has is the trail its own answers leave behind: the suggested action was accepted or undone, the alert was confirmed or closed. We wanted to know how much of the labelled-data recipe one can recover from that single bit per decision, and we measured it on the benchmarks the two model authors picked for their own READMEs rather than on benchmarks of our choosing. Peekaboo leaves the released model untouched and trains a small add-on next to it (a gated LoRA adapter on Vega's decision engine; a copy of Laya's decision head, with or without the encoder) by REINFORCE on the right / wrong reward of a sampled answer, with a running baseline and a KL term that keeps the add-on near the released probabilities. The policy this produces is badly over-confident, so a single temperature is then fitted, also from right / wrong outcomes, using nothing more than the observation that a calibrated model's sampled answer is right as often as it says it is. Over six benchmarks, two Vega sizes, Laya, three seeds and the ablations, 233 runs in all, Peekaboo is ahead of VegaML's own test-time-training head on four of the six benchmarks at both sizes even though that head sees the labels; on DecisionBench it gains 10.4 and 6.7 points where TTT gains 2.2 and 0.8. It stays within a point or two of the same adapter trained on labels. On Laya the result depends on how much of the model we let it touch. Training only the head gives +13.2 on typed-decisions against the +40.2 of Laya's whole-model fine-tune, while unfreezing the encoder under the same one-bit signal gives +32.3 and puts Peekaboo ahead of that fine-tune on the public tasks, DecisionBench and emotion. Without the KL term the accuracy gains shrink or reverse; without the temperature step accuracy is unchanged and calibration error is two to four times worse. All base numbers match their READMEs and one script reruns the grid.

1 Introduction

A decision model, in the sense Vega [16] and Laya [2] use the word, is a model that takes a situation and a typed question and returns one probability per option in a single forward pass. Which of these five tools should the agent call? Is this message abusive? How urgent is this ticket, on a scale of one to four? Both models are built for the regime where an agent or a pipeline asks thousands of such questions a day on local hardware, and both put some care into the probabilities, because the probabilities are what decide which cases get routed to a person.

Each model comes with its author's preferred way of getting better at a new task. For Vega this is test-time training, a small per-task classifier fitted to the frozen model's features from labelled rows, with adapters trained the same way as the heavier option. Laya's recipe is to fine-tune the whole model on labelled rows with its RLCD objective, and the released `laya-typed-decisions` checkpoint is what that recipe produces on Laya's headline benchmark. Labelled rows are the common requirement, and labelled rows are what a deployed model does not get. It gets something weaker and much more plentiful. A proposed tool call was

cancelled by the user. A flagged message turned out, on review, to be fine. An escalated ticket did in fact need escalating. Every one of these events says whether the model was right; none of them says what the right answer was. In the vocabulary of contextual bandits [12, 13] the model sees the reward of the arm it pulled and learns nothing about the arms it did not.

The question we set out to answer is narrow. Can a deployed decision model improve from that one bit per decision, and, if it can, how does the improvement compare with what the model's own authors get from the answer key, on the benchmarks those authors chose? We did not want to pick benchmarks that suit the method, so the test set is the union of the two READMEs (Vega's 43 public tasks, JevBench [23] and DecisionBench [7]; Laya's typed-decisions [14]) plus two ordinary text classification sets with enough rows to study the effect of feedback volume, emotion [21] and AG News [28].

The method, which we call Peekaboo because the model only ever gets to peek at whether it was right, is deliberately plain. We never modify the released weights. Next to them we train a small add-on: for Vega a rank-8 gated LoRA adapter on the input layers of its decision engine, attached with VegaML's own adapter mechanism; for Laya a trainable copy of its two-layer decision head, and in one variant a copy of the encoder as well. The add-on is trained by REINFORCE [27] on the right / wrong reward of an answer sampled from the current policy, with an exponential running mean of recent rewards as the baseline, and a KL penalty towards the released model's probabilities. The penalty matters more than its size suggests; without it one bit per row drives the policy onto a handful of options within a couple of passes. Policy-gradient training also leaves the probabilities far too sharp, and here we had no labels to run temperature scaling on. We fit the temperature from right / wrong feedback instead. A calibrated model's sampled answer is right with exactly the probability it assigned to that answer, so each (answer, outcome) pair on a held-out slice of the feedback is a Bernoulli observation of the temperature-scaled probability, and the temperature that makes the observed record most likely is the one we use. We refer to this as the honesty fix. Finally, for deployments that cannot afford a regression anywhere, a switch compares the add-on and the released model on the same held-out feedback and keeps the add-on only where it was right more often.

Everything is run on Vega 0.8B, Vega 4B and Laya, against each model's own answer-key method, against an answer-key version of our own add-on (same capacity, full labels) as an upper bound, and through every ablation: no KL term, no honesty fix, switch on and off, one add-on per benchmark against one add-on pooled over all of them. Three seeds everywhere, 233 steps in total, scored with VegaML's own vendored benchmark code so that each base model's number can be checked against its README, which it does match.

What we found is roughly this. With one bit per row, Peekaboo is ahead of Vega's labelled-data TTT head on the public tasks, DecisionBench, typed-decisions and AG News, at both model sizes; TTT is ahead on emotion, and clearly so. Peekaboo lands within a point or two of our own adapter trained on the labels, so on Vega the bit is nearly as good as the label. It is also the only thing in our grid that improves Vega 4B on JevBench. On Laya, the head alone is too small a lever: +13.2 on typed-decisions where the whole-model fine-tune gets +40.2. Give Peekaboo the encoder too and it reaches +32.3 there, and on four of the other five benchmarks it does better than Laya's own fine-tune, which specialises hard to typed-decisions and pays for it on JevBench. The two parts we were least sure about, the KL term and the feedback-fitted temperature, both turn out to be load-bearing.

2 Related work

Learning from bandit feedback. Training a structured predictor from the reward of its own outputs goes back at least to bandit structured prediction [22], and Kreutzer et al. [10] showed it works with real human ratings rather than simulated ones. The off-policy branch of this literature learns from logged feedback with importance weights [3, 25]. Peekaboo is the on-policy case, plain REINFORCE with a baseline [27], and what distinguishes it is less the estimator than the starting point: a model that already has a good zero-shot

policy. Our no-engine control (Section 6.6) suggests that this starting point is what lets one bit per row go as far as it does.

KL-regularised fine-tuning. A KL penalty to a reference policy is a fixture of RLHF [1, 17] and of offline RL for dialogue [9]. DPO [20] and KTO [5] fold the same constraint into a supervised objective, KTO from binary desirable / undesirable labels. We use the penalty for the same reason in a far smaller model, and we measure what it buys (Section 6.3).

Calibration. Temperature scaling [6, 18] fits a single scalar on a labelled held-out set. Our version writes the same likelihood over right / wrong outcomes of sampled answers. For a calibrated model $\Pr[\text{right} \mid a] = p(a)$ for whichever answer a was given, so each (answer, outcome) pair is a Bernoulli draw with parameter $p_T(a)$ and the labels are not needed. We report calibration as expected calibration error [15], computed by VegaML's benchmark code.

Adapting small decision models. Test-time training [24] adapts a model to the test distribution; VegaML's `fit()` fits a per-task classifier (PCA followed by logistic regression) on labelled rows and keeps it only when it beats chance in cross-validation. Our Vega add-on is a LoRA adapter [8] in VegaML's own gated form. Laya's RLCD trains encoder and head together on labels with a perturbation policy gradient on proper scoring rules.

3 The two decision models

Vega. Vega [16] reads the situation, the question and the options with a frozen Qwen3.5 backbone [19] of 0.8B or 4B parameters, pools hidden states from two intermediate layers, and passes them to what its authors call a physics engine (14.3M parameters at 0.8B, 30.9M at 4B). Each option becomes an energy well in a 64-dimensional landscape; a particle is dropped and allowed to settle; the output is $P(a) = \text{softmax}(-E_a/\tau(s))$ with a per-question temperature read off the particle's trajectory. The backbone was never trained, only the engine and its gated adapters. Vega's own routes to a better model on a task are the TTT head and adapters, both from labelled rows.

Laya. Laya [2] is a ModernBERT-large encoder [26] of 421M parameters, trained as part of Laya, whose input holds the situation, the question and every option, each option introduced by a special marker token. A two-layer transformer decision head scores the marker positions, the scores are divided by a temperature chosen per (question type, option-count bucket), and a softmax gives the probabilities. Training is by RLCD. Laya's route to a better model is a whole-model fine-tune on labelled rows; `laya-typed-decisions` is that fine-tune on the training split of typed-decisions.

The two models expose the same interface, a typed question in and a probability per option out, which is what let us use one method and one harness for both. They differ in almost everything else, including how much of each model our add-on can reach, and that difference ends up mattering (Section 6.5).

4 Peekaboo

4.1 Setting

A decision model is a policy $\pi_0(a \mid s)$ over the options a of a typed question s. In deployment it answers a stream of questions; for each one it, or the system around it, commits to an answer a, and some time later a reward $r \in \{0, 1\}$ arrives. The correct option is never revealed. From a pool of (question, answer,

reward) records we want a policy π that is more often right and whose probabilities can still be trusted. In the experiments the reward is computed from the benchmark's gold answer and the answer is sampled from the current policy, so the feedback is on-policy and never wrong. That is an idealisation and we return to it in Section 8.

4.2 The add-on

The released model is never changed. On Vega we attach a single gated LoRA adapter [8] of rank 8 to the input layers of the engine with VegaML's `attach_adapter`; it has 409k parameters at 0.8B and 857k at 4B, about 2.9% and 2.8% of the engine. Its gate starts open and the adapters that ship with Vega keep working alongside it. The policy is Vega's usual calibrated output computed from the adapted energies with the shipped per-question temperature. On Laya we make a trainable copy of the decision head (type embedding, the two transformer layers, the scorer) and run the frozen encoder once per row, caching its output. In the encoder-unfrozen variant we copy the encoder as well and train it at Laya's own encoder learning rate of 2.5×10^{-5}, in micro-batches of 8 for memory, so that Peekaboo is allowed to move as much of Laya as Laya's own fine-tune moves.

4.3 The update

For a batch of questions the policy samples one answer per question, $a \sim \pi(\cdot \mid s)$, receives $r = \mathbf{1}[a \text{ is right}]$, and minimises

$$\mathcal{L} \;=\; -\,(r - b)\,\log \pi(a \mid s) \;+\; \beta\,\mathrm{KL}\big(\pi(\cdot \mid s)\,\|\,\pi_0(\cdot \mid s)\big), \qquad \beta = 0.1, \tag{1}$$

where b is an exponential running mean of recent rewards, $b \leftarrow 0.9\,b + 0.1\,\bar{r}$, started at 0.5. The first term is REINFORCE with a baseline: an answer that did better than the recent average becomes more likely in similar situations, one that did worse becomes less likely. The second term pulls the policy back towards the released model. In our pilot runs without it the negative log-likelihood climbed to 7.5 as the policy collapsed onto a few options; Section 6.3 puts a number on the cost for each benchmark. Training is 8 passes over the feedback rows in batches of 32 with Adam, learning rate 10^{-3} for the Vega adapter and 10^{-4} for the Laya head (Laya's own head rate), gradient norm clipped at 1.

4.4 The honesty fix

Policy-gradient training sharpens the probabilities. Before any correction the trained add-ons have an ECE somewhere between 0.17 and 0.35 (Table 6), which would make the probabilities useless for routing. Temperature scaling [6] is the standard remedy and it wants labels. We do not have labels, but we do have the following. If a model is calibrated, the answer a it samples is right with probability exactly $p(a)$. So on a held-out calibration split, 20% of each task's feedback rows, we sample $a \sim \pi$, observe r, and choose

$$T^{\star} = \arg\max_{T} \sum_{i} \Big[r_i \log p_T(a_i \mid s_i) + (1 - r_i) \log\big(1 - p_T(a_i \mid s_i)\big) \Big], \qquad p_T = \mathrm{softmax}(\mathrm{logits}/T), \tag{2}$$

by searching a grid of 61 values of T between 0.25 and 32. The temperatures that come out lie between about 3 and 9 (Figure 5), which is a measure of how far policy gradient had pushed the probabilities. The identity does not care who chose the answer, so the same fit can be run on logged feedback about the released model's own answers.

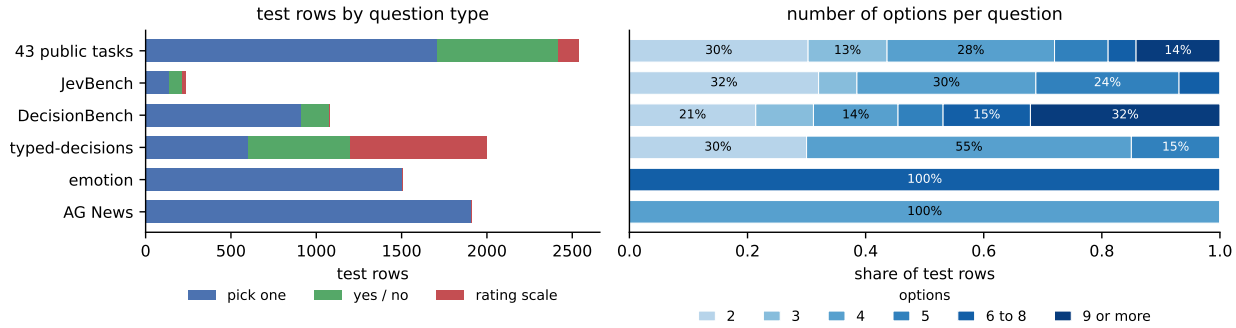


Figure 1: The six benchmarks' test rows, by question type (left) and by the number of options a question offers (right, as a share of that benchmark's rows). Emotion and AG News always offer the same number of options, six and four; the public menu and DecisionBench include questions with dozens to hundreds.

4.5 The switch

On the same calibration split we sample one answer from the adapted policy and one from the released model for each row, score both, and keep the add-on only if it was right more often. Nothing is needed beyond the feedback already collected. Every Peekaboo result is reported with and without this switch, because it has a cost: on a small calibration fold the comparison is noisy and sometimes throws away a real gain.

4.6 Pooled Peekaboo

The default trains one add-on per benchmark from that benchmark's feedback. The pooled variant trains one add-on per model from the feedback of all five benchmarks that have a training split and evaluates it on all six. JevBench has no training split, so for the pooled add-on it is a pure transfer test.

4.7 Cost

The backbone is untouched and inference cost changes only by the add-on. One row at a time on a laptop CPU, Vega's engine takes 6.3 ms at the median when released and 6.5 ms with the adapter at 0.8B, 11.4 against 12.3 ms at 4B. The backbone itself takes roughly 75 to 245 ms on a T4. Training on cached features takes between one and fifteen minutes per benchmark and seed on an L40S; the Laya encoder-unfrozen runs took 4 to 15 minutes each.

5 Experimental setup

5.1 Benchmarks

We use the benchmarks the two READMEs report and nothing else from their domain, plus the two classification sets (Table 1, Figure 1). Rows are read, asked and scored with VegaML's benchmark harness, vendored without modification under its Apache-2.0 licence, so each number can be set next to Vega's README.

For DecisionBench the test set is the README's 1,075-row slice (25 rows per task, the notebook's seed) and the feedback pool is a 100-per-task superset of the same tasks, as in our earlier DecisionBench study. For the public menu the harness's 60 evaluation rows per task are the test set and its 50 training rows per task are the feedback pool; those are the same 50 rows VegaML's TTT trains on. For typed-decisions we use the public LocalLLaMA release, which is what Laya reports on. Vega's README quotes a typed-decisions

Table 1: The six benchmarks. "Feedback rows" are the rows whose right / wrong signal Peekaboo may learn from (80%) and calibrate on (20%); no test row is ever used for learning. Question types are pick one (choice), yes / no, and rating scale (score). JevBench has no training split, so Peekaboo learns by 5-fold cross-fitting with paraphrase groups kept in one fold.

benchmark	reported by	test rows	feedback rows	what it contains	types (test)
43 public tasks	Vega	2,535	2,134	43 public datasets as typed questions, up to 60 test rows each: intent (77 and 150 options), topic, safety, sentiment, entailment, science and medical QA, law, reading, code; macro-averaged over tasks	1,709 / 707 / 119
JevBench	Vega	231	(5-fold)	rules, policies and judgment calls in 18 families; tiers hard 111 / original 72 / easy 48; paraphrase groups	139 / 74 / 18
DecisionBench	Vega	1,075	3,205	43 tasks in 28 domains (agent control, coding agents, finance, law, privacy, robotics, …); candidate selection 750, binary 225, ordinal 100 (asked as pick-one); up to 255 options	913 / 162 / 0
typed-decisions	Laya	2,000	6,000	400 test and 1,200 training JSON cases (tickets, invoices, agent traces, security alerts), five decisions each, four workflows of 500 test decisions	600 / 600 / 800
emotion	(ours)	1,503	4,000	six emotions of a tweet	1,503 / 0 / 0
AG News	(ours)	1,912	4,000	four news sections	1,912 / 0 / 0

number too, but on six unpublished workflows that nobody outside VegaML can rerun, so Vega's own typed-decisions figure is not compared here.

5.2 Protocol

Every method learns from feedback rows (or, for the answer-key methods, their labels) and is scored on test rows. On JevBench the 5-fold cross-fitting keeps all paraphrases of a question in one fold. The feedback rows of each task are split 80 / 20 into training and calibration rows with a fixed hash. Stochastic methods run with seeds 0, 1 and 2 and we report mean ± s.d. Gains are paired: the per-row correctness of a method, averaged over its seeds, minus the base model's on the same rows, with a 95% bootstrap confidence interval over rows [4], resampled within task for the macro-averaged public menu. Accuracy, ECE (15 equal-width bins, as in the README), NLL, skill, macro-F1, JevBench's tiers and paraphrase consistency, DecisionBench's domain slices and typed-decisions' soft accuracy all come from the harness's `score_metrics`.

5.3 Baselines

- **Base model.** Vega 0.8B, Vega 4B and Laya as released. Each is first replayed: we run the frozen model on the cached inputs and check its probabilities against the official prediction call (largest absolute difference 5×10^{-5} for Laya, 0 for Vega) and its scores against the README.
- **Their improvement, from the answer key.** For Vega, VegaML's TTT head (`fit()` with default settings) trained on the labelled feedback rows of each task. For Laya, the released `laya-typed-decisions` checkpoint, which is Laya's whole-model fine-tune on typed-decisions' labelled training split, scored on all six benchmarks.

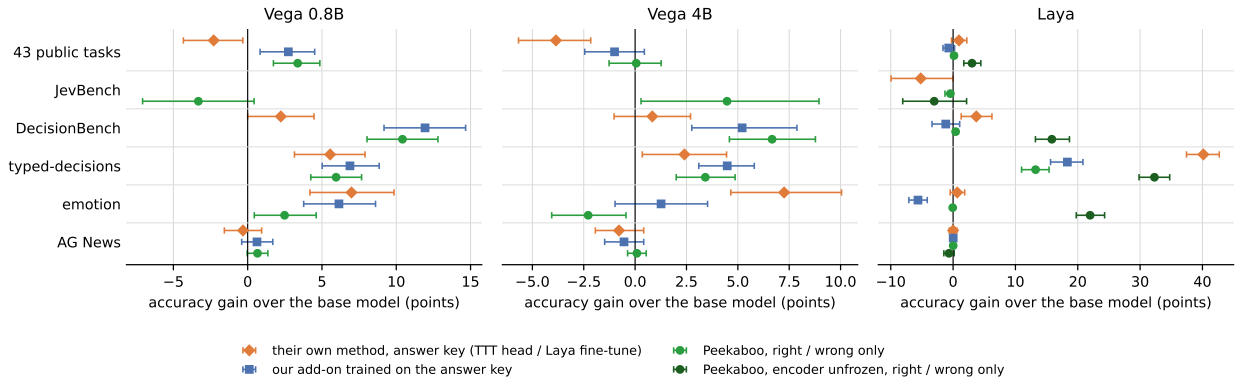


Figure 2: How much each method improves on the base model. Each row is a benchmark, each marker a method's accuracy gain over the base model on that benchmark in points, with its paired 95% bootstrap interval; a marker to the right of the vertical line is an improvement. Orange diamonds: the model's own answer-key method (VegaML's TTT head for Vega, the released typed-decisions fine-tune for Laya). Blue squares: our add-on trained on the answer key. Green circles: Peekaboo, from right / wrong only; dark green, on Laya, with the encoder unfrozen. JevBench has no training split, so only Peekaboo is defined there.

- **Answer-key add-on.** The same adapter (Vega) or head copy (Laya) trained with cross-entropy on the labels of the same rows, with the same KL term, followed by ordinary label-based temperature scaling on the same calibration rows. Same capacity as Peekaboo, full information.
- **Peekaboo and its ablations.** No KL term ($\beta = 0$); no honesty fix (the trained add-on at $T = 1$); with the switch; pooled; and on Laya the encoder-unfrozen variant.
- **No-engine control** (DecisionBench study only). A bilinear scorer on the same frozen features, trained with the same feedback, KL term and calibration, without Vega's engine.

5.4 Compute and reproducibility

The grid, 6 benchmarks × 3 models × methods × 3 seeds for 233 steps, ran on one NVIDIA L40S in about five GPU-hours from cached backbone features. A single script, `experiments/peekaboo_remaining/run.py`, runs every step, skips the finished ones, retries failures and writes the tables; results, logs and the tables in this paper are in the repository.[1] Section 6.6 ran on a laptop CPU and was reproduced independently on a second machine (Appendix A).

6 Results

6.1 Main comparison

Table 3 and Figure 2 carry the headline comparison, with absolute accuracies in Table 2 and calibration in Table 4.

Against Vega's own method. On DecisionBench Peekaboo gains +10.4 points [+8.0, +12.8] at 0.8B and +6.7 [+4.6, +8.8] at 4B. VegaML's TTT head, trained on the labels of the same rows, gains +2.2 and +0.8. On the public menu Peekaboo gains +3.4 and +0.1 while TTT loses 2.3 and 3.9 points. The per-task table (Appendix B) shows where the TTT loss comes from: a logistic head fitted to 50 labelled rows is quite capable of overturning Vega's zero-shot answer on tasks where the zero-shot answer was the better one, and

[1] https://github.com/abhijithneilabraham/peekaboo

Table 2: Accuracy. Mean over 3 seeds (± s.d.) where a method is stochastic. JevBench has no training split: Peekaboo learns by 5-fold cross-fitting (paraphrase groups kept together), pooled Peekaboo never sees it, and VegaML's TTT and the answer-key methods are not defined for it.

model	method	learns from	public	JevBench	DecisionB.	typed-dec.	emotion	AG News
Vega 0.8B	base model	none	0.556	0.550	0.509	0.665	0.568	0.901
	VegaML's TTT head	answer key	0.533	·	0.531	0.721	0.638	0.897
	answer-key adapter	answer key	0.584 ±0.005	·	0.628 ±0.008	0.734 ±0.000	0.630 ±0.011	0.907 ±0.001
	Peekaboo	right / wrong	0.590 ±0.013	0.517 ±0.018	0.613 ±0.011	0.725 ±0.012	0.593 ±0.009	0.907 ±0.002
	pooled Peekaboo	right / wrong	0.596 ±0.006	0.548 ±0.011	0.606 ±0.011	0.720 ±0.015	0.567 ±0.040	0.911 ±0.005
Vega 4B	base model	none	0.715	0.667	0.631	0.730	0.579	0.927
	VegaML's TTT head	answer key	0.677	·	0.639	0.753	0.651	0.919
	answer-key adapter	answer key	0.705 ±0.005	·	0.683 ±0.011	0.774 ±0.006	0.592 ±0.022	0.921 ±0.003
	Peekaboo	right / wrong	0.716 ±0.007	0.711 ±0.004	0.697 ±0.006	0.764 ±0.006	0.556 ±0.009	0.928 ±0.002
	pooled Peekaboo	right / wrong	0.720 ±0.005	0.687 ±0.002	0.682 ±0.008	0.744 ±0.018	0.544 ±0.009	0.932 ±0.003
Laya	base model	none	0.542	0.589	0.401	0.361	0.597	0.930
	laya-typed-decisions (Laya's fine-tune)	answer key	0.551	0.537	0.438	0.762	0.604	0.930
	answer-key head	answer key	0.535 ±0.004	·	0.389 ±0.009	0.544 ±0.005	0.540 ±0.032	0.930 ±0.000
	Peekaboo, head only	right / wrong	0.543 ±0.001	0.584 ±0.000	0.405 ±0.002	0.493 ±0.008	0.596 ±0.001	0.930 ±0.000
	Peekaboo, encoder unfrozen	right / wrong	0.572 ±0.007	0.558 ±0.011	0.559 ±0.005	0.684 ±0.021	0.817 ±0.003	0.923 ±0.004
	pooled Peekaboo, head only	right / wrong	0.538 ±0.002	0.569 ±0.007	0.398 ±0.006	0.475 ±0.004	0.594 ±0.002	0.930 ±0.000

Table 3: Gain over the base model, in accuracy points. Mean over 3 seeds (± s.d.) where a method is stochastic. Each gain is paired on the same rows against the model's own base; the CIs are in Table 5 and Figure 2. JevBench has no training split: Peekaboo learns by 5-fold cross-fitting (paraphrase groups kept together), pooled Peekaboo never sees it, and VegaML's TTT and the answer-key methods are not defined for it.

model	method	learns from	public	JevBench	DecisionB.	typed-dec.	emotion	AG News
Vega 0.8B	base model	none	–	–	–	–	–	–
	VegaML's TTT head	answer key	-2.3	·	+2.2	+5.5	+7.0	-0.3
	answer-key adapter	answer key	+2.7	·	+11.9	+6.9	+6.1	+0.6
	Peekaboo	right / wrong	+3.4	-3.3	+10.4	+5.9	+2.5	+0.7
	pooled Peekaboo	right / wrong	+4.0	-0.1	+9.7	+5.4	-0.2	+1.1
Vega 4B	base model	none	–	–	–	–	–	–
	VegaML's TTT head	answer key	-3.9	·	+0.8	+2.4	+7.3	-0.8
	answer-key adapter	answer key	-1.0	·	+5.2	+4.5	+1.3	-0.5
	Peekaboo	right / wrong	+0.1	+4.5	+6.7	+3.4	-2.3	+0.1
	pooled Peekaboo	right / wrong	+0.4	+2.0	+5.1	+1.4	-3.5	+0.5
Laya	base model	none	–	–	–	–	–	–
	laya-typed-decisions (Laya's fine-tune)	answer key	+0.9	-5.2	+3.7	+40.2	+0.7	+0.0
	answer-key head	answer key	-0.7	·	-1.2	+18.3	-5.6	+0.0
	Peekaboo, head only	right / wrong	+0.1	-0.4	+0.4	+13.2	-0.1	+0.0
	Peekaboo, encoder unfrozen	right / wrong	+3.0	-3.0	+15.8	+32.3	+22.0	-0.7
	pooled Peekaboo, head only	right / wrong	-0.4	-2.0	-0.2	+11.4	-0.2	+0.0

Table 4: Expected calibration error (lower is better). Mean over 3 seeds (± s.d.) where a method is stochastic. JevBench has no training split: Peekaboo learns by 5-fold cross-fitting (paraphrase groups kept together), pooled Peekaboo never sees it, and VegaML's TTT and the answer-key methods are not defined for it.

model	method	learns from	public	JevBench	DecisionB.	typed-dec.	emotion	AG News
Vega 0.8B	base model	none	0.154	0.173	0.105	0.063	0.033	0.021
	VegaML's TTT head	answer key	0.208	·	0.113	0.139	0.039	0.043
	answer-key adapter	answer key	0.182 ±0.003	·	0.071 ±0.014	0.079 ±0.008	0.059 ±0.013	0.022 ±0.004
	Peekaboo	right / wrong	0.203 ±0.007	0.092 ±0.018	0.055 ±0.015	0.077 ±0.019	0.070 ±0.025	0.016 ±0.004
	pooled Peekaboo	right / wrong	0.193 ±0.024	0.108 ±0.034	0.066 ±0.009	0.062 ±0.031	0.068 ±0.023	0.019 ±0.005
Vega 4B	base model	none	0.141	0.158	0.099	0.050	0.054	0.027
	VegaML's TTT head	answer key	0.173	·	0.112	0.119	0.043	0.041
	answer-key adapter	answer key	0.181 ±0.006	·	0.051 ±0.009	0.056 ±0.007	0.038 ±0.010	0.014 ±0.001
	Peekaboo	right / wrong	0.172 ±0.003	0.098 ±0.008	0.049 ±0.009	0.044 ±0.013	0.035 ±0.012	0.018 ±0.003
	pooled Peekaboo	right / wrong	0.176 ±0.015	0.134 ±0.038	0.057 ±0.019	0.050 ±0.005	0.044 ±0.009	0.016 ±0.006
Laya	base model	none	0.212	0.113	0.174	0.183	0.306	0.012
	laya-typed-decisions (Laya's fine-tune)	answer key	0.166	0.101	0.045	0.206	0.185	0.106
	answer-key head	answer key	0.143 ±0.001	·	0.066 ±0.006	0.035 ±0.008	0.072 ±0.016	0.010 ±0.002
	Peekaboo, head only	right / wrong	0.157 ±0.004	0.063 ±0.012	0.071 ±0.006	0.052 ±0.005	0.086 ±0.014	0.018 ±0.004
	Peekaboo, encoder unfrozen	right / wrong	0.152 ±0.006	0.087 ±0.012	0.062 ±0.009	0.062 ±0.019	0.035 ±0.007	0.030 ±0.007
	pooled Peekaboo, head only	right / wrong	0.164 ±0.007	0.096 ±0.019	0.067 ±0.015	0.052 ±0.016	0.087 ±0.006	0.016 ±0.003

Table 5: Peekaboo head to head, in accuracy points with a paired 95% bootstrap CI. Positive means Peekaboo is better. "their improvement" is VegaML's TTT head for Vega and the released laya-typed-decisions fine-tune for Laya, both trained with the answer key.

model	benchmark	vs. base	vs. their improvement	vs. answer-key adapter
Vega 0.8B	43 public tasks	+3.4 [+1.7, +4.9]	+5.7 [+3.8, +7.5]	+0.6 [-0.6, +1.9]
	JevBench	-3.3 [-7.1, +0.4]	–	–
	DecisionBench	+10.4 [+8.0, +12.8]	+8.2 [+5.8, +10.7]	-1.5 [-3.4, +0.2]
	typed-decisions	+5.9 [+4.2, +7.7]	+0.4 [-1.5, +2.5]	-0.9 [-1.9, +0.1]
	emotion	+2.5 [+0.4, +4.6]	-4.5 [-6.6, -2.3]	-3.7 [-5.3, -2.1]
	AG News	+0.7 [-0.0, +1.4]	+1.0 [-0.1, +2.1]	+0.0 [-0.8, +0.8]
Vega 4B	43 public tasks	+0.1 [-1.3, +1.3]	+3.9 [+2.2, +5.6]	+1.1 [+0.0, +2.1]
	JevBench	+4.5 [+0.3, +9.0]	–	–
	DecisionBench	+6.7 [+4.6, +8.8]	+5.8 [+3.7, +7.8]	+1.5 [-0.2, +3.1]
	typed-decisions	+3.4 [+2.0, +4.9]	+1.0 [-0.7, +2.7]	-1.1 [-1.9, -0.2]
	emotion	-2.3 [-4.1, -0.4]	-9.5 [-11.6, -7.3]	-3.5 [-5.4, -1.7]
	AG News	+0.1 [-0.4, +0.5]	+0.9 [-0.1, +1.9]	+0.6 [-0.2, +1.4]
Laya, head only	43 public tasks	+0.1 [-0.3, +0.5]	-0.8 [-2.1, +0.5]	+0.8 [+0.0, +1.7]
	JevBench	-0.4 [-1.3, +0.0]	+4.8 [-0.4, +9.5]	–
	DecisionBench	+0.4 [+0.0, +0.8]	-3.3 [-5.8, -0.9]	+1.6 [-0.6, +3.7]
	typed-decisions	+13.2 [+11.0, +15.4]	-26.9 [-28.9, -24.8]	-5.1 [-7.0, -3.4]
	emotion	-0.1 [-0.4, +0.2]	-0.7 [-1.9, +0.4]	+5.6 [+4.1, +7.0]
	AG News	+0.0 [-0.0, +0.1]	+0.0 [-0.6, +0.6]	+0.0 [-0.2, +0.2]
Laya, encoder unfrozen	43 public tasks	+3.0 [+1.7, +4.4]	+2.1 [+0.7, +3.6]	+3.7 [+2.4, +5.1]
	JevBench	-3.0 [-8.1, +2.2]	+2.2 [-3.3, +7.5]	–
	DecisionBench	+15.8 [+13.2, +18.7]	+12.1 [+9.6, +14.8]	+17.0 [+14.4, +19.5]
	typed-decisions	+32.3 [+29.8, +34.8]	-7.9 [-9.4, -6.3]	+14.0 [+12.1, +15.8]
	emotion	+22.0 [+19.8, +24.3]	+21.3 [+19.1, +23.6]	+27.6 [+25.6, +29.7]
	AG News	-0.7 [-1.5, +0.2]	-0.7 [-1.4, +0.1]	-0.7 [-1.5, +0.2]

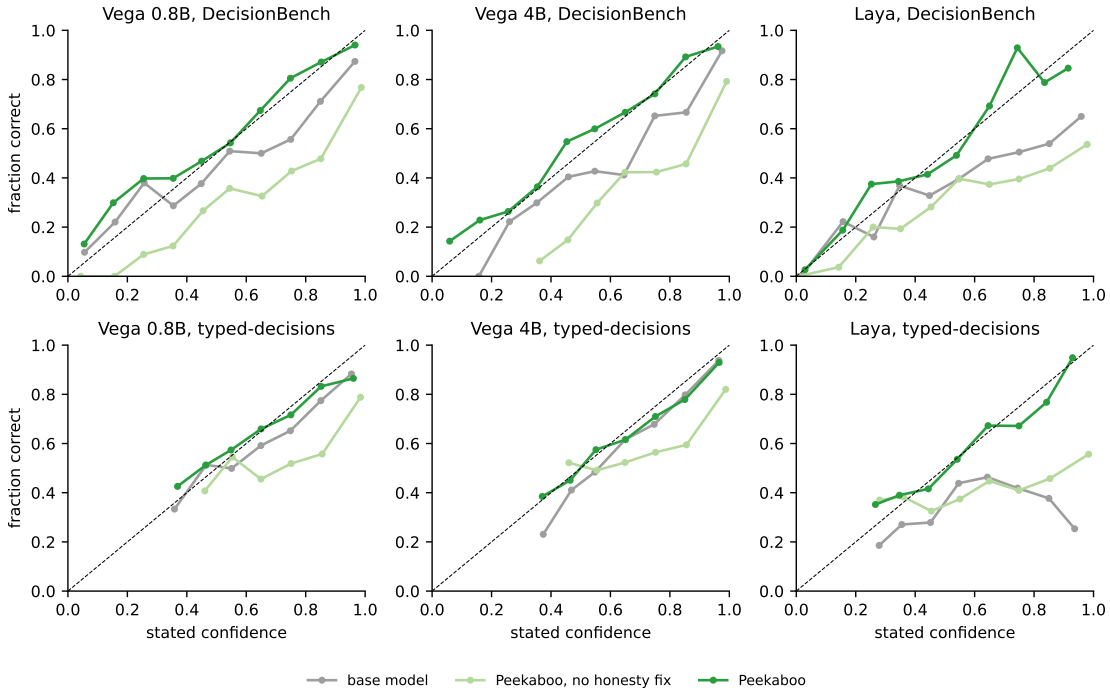


Figure 3: Reliability diagrams (10 bins on the confidence of the predicted option) on DecisionBench and typed-decisions. Grey is the base model; light green the trained Peekaboo add-on before the honesty fix, which is over-confident everywhere; dark green Peekaboo after it.

`fit()` only checks that the head beats chance, not that it beats the zero-shot answer. On typed-decisions the two are close at 0.8B (+5.9 against +5.5) and Peekaboo is ahead at 4B (+3.4 against +2.4). On AG News neither moves the base model by more than a point. Emotion goes the other way, and not narrowly: TTT gains +7.0 and +7.3 where Peekaboo gets +2.5 at 0.8B and loses 2.3 at 4B. A six-way single-label task with 4,000 labelled rows is where logistic regression on good features is at its best, and one bit per row is a poor substitute for the label there.

Against the same add-on with the answer key. The answer-key adapter has the capacity of Peekaboo and all the information. The paired difference, Peekaboo minus adapter, is +0.7 [−0.9, +2.2] on the public menu, −1.5 [−3.5, +0.5] on DecisionBench, −0.9 [−2.5, +0.6] on typed-decisions and +0.1 on AG News at 0.8B; at 4B it is +1.1, +1.5, −1.1 and +0.6 (Table 5). Emotion is again the exception, at −3.6 for both sizes. Elsewhere the intervals straddle zero, and we read this as the bit being worth nearly as much as the label once the model it feeds already knows roughly what it is doing.

JevBench. JevBench has 231 rows, no training split, and asks for judgment under paraphrase. Only Peekaboo and the base model are defined on it, since TTT and the answer-key methods need labelled training rows and the cross-fitted Peekaboo only needs rewards on the other folds. At 4B it is the one improvement we found on this benchmark: +4.5 [+0.3, +9.0]. At 0.8B it loses 3.3 points [−7.1, +0.4]. The switch (Section 6.3) brings that to −0.6 and pooling (Section 6.4) to −0.1, which is the kind of result that is easy to over-read; the JevBench intervals are wide and we would not build much on them.

6.2 Calibration

Table 4 and Figure 3 show the calibration side. Policy-gradient training over-sharpens: before the honesty fix the add-on's ECE is 0.24 on DecisionBench at 0.8B, 0.22 at 4B, and 0.20 and 0.17 on typed-decisions. The feedback-fitted temperature brings those to 0.055, 0.049, 0.077 and 0.044. On DecisionBench that is below the base model (0.105 and 0.099) and below the answer-key adapter with label-based temperature scaling (0.071 and 0.051); on typed-decisions it is about level with base (0.063 and 0.050). The public menu is the case that does not work. Every trained method, Peekaboo included, is worse calibrated there than base Vega (0.203 against 0.154 at 0.8B), and we think the reason is simply that one temperature for 43 tasks whose option counts run from 2 to 151 is too coarse an instrument. On Laya the honesty fix is what makes the trained head usable at all. Base Laya is poorly calibrated on DecisionBench (0.174) and emotion (0.306), and Peekaboo lands both between 0.07 and 0.09.

6.3 Ablations

Figure 4 and Table 6 take the method apart.

The KL term. With $\beta = 0$ the gain drops on five of the six benchmarks at both sizes. At 0.8B it goes from +3.4 to +1.2 on the public menu, −3.3 to −4.0 on JevBench, +5.9 to +3.3 on typed-decisions and +2.5 to −3.9 on emotion; at 4B from +6.7 to +5.0 on DecisionBench, +3.4 to +1.1 on typed-decisions and −2.3 to −5.8 on emotion. AG News, where the base model is already at 0.90 to 0.93, does not care either way. The KL term also keeps calibration closer to base on the public menu. We chose $\beta = 0.1$ in a pilot where $\beta = 1$ was stable but kept only about a third of the gain, and we never went back to retune it; it is possible a different value would do better on emotion.

The honesty fix. A temperature does not change the argmax, so accuracy with and without it is identical by construction (Table 6). ECE without it is 1.6 to 4.4 times higher: on the public menu 0.203 against 0.328, on DecisionBench 0.055 against 0.241, on emotion 0.070 against 0.251, all at 0.8B. Anything downstream that reads the probabilities rather than the argmax, a routing threshold or a conformal set, needs this step.

The switch. The switch never turns a loss into a gain beyond base, and it sometimes removes a gain. At 4B it takes DecisionBench from +6.7 to +4.5 and JevBench from +4.5 to +0.9, because a sampled comparison on a 42-row calibration fold is a noisy thing. Where Peekaboo is hurting it does its job: JevBench at 0.8B goes from −3.3 to −0.6, and the pooled add-on's emotion loss at 4B from −3.5 to −0.8. Figure 5 shows how often it fired. A deployment with more than 42 calibration rows would get a steadier switch than we did.

6.4 Pooled Peekaboo

One add-on fed by every benchmark's feedback (Figure 6) comes out mixed. At 0.8B it is a little better on the public menu (+4.0 against +3.4) and AG News (+1.1 against +0.7), a little worse on DecisionBench, typed-decisions and emotion, and it removes the JevBench loss (−0.1 against −3.3) without having seen a JevBench row. That last part is the one we can least account for. Feedback from roughly 10,000 unrelated decisions seems to teach the adapter something general about how Vega mis-weighs options, enough to undo the damage the per-benchmark adapter did on JevBench. At 4B, which was winning JevBench on its own, pooling is slightly worse almost everywhere (+2.0 against +4.5 on JevBench, +5.1 against +6.7 on DecisionBench). On Laya's frozen-encoder head it is uniformly a bit worse. We offer pooling as the option for a model that has to stay good on tasks it gets no feedback about, and would not make it the default.

Table 6: Every variant and ablation: accuracy (top) and expected calibration error (bottom), mean over 3 seeds. "switch": the add-on is used only where held-out right / wrong feedback says it beats the base model. "no honesty fix": the trained add-on without the feedback-fitted temperature. "no KL term": $\beta = 0$.

model	variant	public	JevBench	DecisionB.	typed-dec.	emotion	AG News
accuracy							
Vega 0.8B	base model	0.556	0.550	0.509	0.665	0.568	0.901
	Peekaboo	0.590 ±0.013	0.517 ±0.018	0.613 ±0.011	0.725 ±0.012	0.593 ±0.009	0.907 ±0.002
	Peekaboo + switch	0.578 ±0.020	0.544 ±0.004	0.613 ±0.011	0.711 ±0.032	0.593 ±0.009	0.907 ±0.002
	Peekaboo without the honesty fix	0.590 ±0.013	0.517 ±0.018	0.613 ±0.011	0.725 ±0.012	0.593 ±0.009	0.907 ±0.002
	Peekaboo without the KL term ($\beta = 0$)	0.568 ±0.021	0.509 ±0.025	0.612 ±0.007	0.698 ±0.020	0.529 ±0.020	0.908 ±0.001
	pooled Peekaboo (one add-on, all suites' feedback)	0.596 ±0.006	0.548 ±0.011	0.606 ±0.011	0.720 ±0.015	0.567 ±0.040	0.911 ±0.005
	pooled Peekaboo + switch	0.568 ±0.016	0.548 ±0.011	0.606 ±0.011	0.708 ±0.031	0.567 ±0.040	0.911 ±0.005
	pooled Peekaboo without the honesty fix	0.596 ±0.006	0.548 ±0.011	0.606 ±0.011	0.720 ±0.015	0.567 ±0.040	0.911 ±0.005
Vega 4B	base model	0.715	0.667	0.631	0.730	0.579	0.927
	Peekaboo	0.716 ±0.007	0.711 ±0.004	0.697 ±0.006	0.764 ±0.006	0.556 ±0.009	0.928 ±0.002
	Peekaboo + switch	0.713 ±0.003	0.675 ±0.006	0.676 ±0.032	0.764 ±0.006	0.556 ±0.009	0.927 ±0.000
	Peekaboo without the honesty fix	0.716 ±0.007	0.711 ±0.004	0.697 ±0.006	0.764 ±0.006	0.556 ±0.009	0.928 ±0.002
	Peekaboo without the KL term ($\beta = 0$)	0.707 ±0.008	0.704 ±0.011	0.681 ±0.002	0.741 ±0.007	0.521 ±0.022	0.930 ±0.002
	pooled Peekaboo (one add-on, all suites' feedback)	0.720 ±0.005	0.687 ±0.002	0.682 ±0.008	0.744 ±0.018	0.544 ±0.009	0.932 ±0.003
	pooled Peekaboo + switch	0.715 ±0.000	0.687 ±0.002	0.651 ±0.029	0.741 ±0.017	0.571 ±0.011	0.927 ±0.000
	pooled Peekaboo without the honesty fix	0.720 ±0.005	0.687 ±0.002	0.682 ±0.008	0.744 ±0.018	0.544 ±0.009	0.932 ±0.003
Laya	base model	0.542	0.589	0.401	0.361	0.597	0.930
	Peekaboo	0.543 ±0.001	0.584 ±0.000	0.405 ±0.002	0.493 ±0.008	0.596 ±0.001	0.930 ±0.000
	Peekaboo + switch	0.542 ±0.000	0.586 ±0.002	0.401 ±0.000	0.493 ±0.008	0.597 ±0.000	0.930 ±0.000
	Peekaboo without the honesty fix	0.543 ±0.001	0.584 ±0.000	0.405 ±0.002	0.493 ±0.008	0.596 ±0.001	0.930 ±0.000
	Peekaboo without the KL term ($\beta = 0$)	0.539 ±0.006	0.584 ±0.000	0.403 ±0.001	0.463 ±0.009	0.586 ±0.009	0.930 ±0.001
	Peekaboo, encoder unfrozen	0.572 ±0.007	0.558 ±0.011	0.559 ±0.005	0.684 ±0.021	0.817 ±0.003	0.923 ±0.004
	Peekaboo, encoder unfrozen + switch	0.555 ±0.019	0.579 ±0.017	0.559 ±0.005	0.684 ±0.021	0.817 ±0.003	0.930 ±0.000
	Peekaboo, encoder unfrozen, no honesty fix	0.572 ±0.007	0.558 ±0.011	0.559 ±0.005	0.684 ±0.021	0.817 ±0.003	0.923 ±0.004
	pooled Peekaboo (one add-on, all suites' feedback)	0.538 ±0.002	0.569 ±0.007	0.398 ±0.006	0.475 ±0.004	0.594 ±0.002	0.930 ±0.000
	pooled Peekaboo + switch	0.542 ±0.000	0.569 ±0.007	0.401 ±0.000	0.475 ±0.004	0.597 ±0.000	0.930 ±0.000
	pooled Peekaboo without the honesty fix	0.538 ±0.002	0.569 ±0.007	0.398 ±0.006	0.475 ±0.004	0.594 ±0.002	0.930 ±0.000
calibration error (ECE)							
Vega 0.8B	base model	0.154	0.173	0.105	0.063	0.033	0.021
	Peekaboo	0.203 ±0.007	0.092 ±0.018	0.055 ±0.015	0.077 ±0.019	0.070 ±0.025	0.016 ±0.004
	Peekaboo + switch	0.184 ±0.021	0.152 ±0.003	0.055 ±0.015	0.063 ±0.001	0.070 ±0.025	0.016 ±0.004
	Peekaboo without the honesty fix	0.328 ±0.018	0.267 ±0.008	0.241 ±0.010	0.198 ±0.019	0.251 ±0.063	0.075 ±0.003
	Peekaboo without the KL term ($\beta = 0$)	0.213 ±0.016	0.097 ±0.014	0.082 ±0.024	0.052 ±0.006	0.048 ±0.009	0.018 ±0.003
	pooled Peekaboo (one add-on, all suites' feedback)	0.193 ±0.024	0.108 ±0.034	0.066 ±0.009	0.062 ±0.031	0.068 ±0.023	0.019 ±0.005
	pooled Peekaboo + switch	0.157 ±0.004	0.108 ±0.034	0.066 ±0.009	0.048 ±0.011	0.068 ±0.023	0.019 ±0.005
	pooled Peekaboo without the honesty fix	0.267 ±0.038	0.308 ±0.022	0.232 ±0.051	0.154 ±0.018	0.311 ±0.110	0.073 ±0.014
Vega 4B	base model	0.141	0.158	0.099	0.050	0.054	0.027
	Peekaboo	0.172 ±0.003	0.098 ±0.008	0.049 ±0.009	0.044 ±0.013	0.035 ±0.012	0.018 ±0.003
	Peekaboo + switch	0.150 ±0.013	0.128 ±0.005	0.064 ±0.026	0.044 ±0.013	0.035 ±0.012	0.027 ±0.000
	Peekaboo without the honesty fix	0.235 ±0.003	0.167 ±0.012	0.223 ±0.009	0.171 ±0.004	0.311 ±0.026	0.049 ±0.016
	Peekaboo without the KL term ($\beta = 0$)	0.182 ±0.011	0.099 ±0.005	0.075 ±0.027	0.049 ±0.016	0.052 ±0.004	0.016 ±0.002
	pooled Peekaboo (one add-on, all suites' feedback)	0.176 ±0.015	0.134 ±0.038	0.057 ±0.019	0.050 ±0.005	0.044 ±0.009	0.016 ±0.006
	pooled Peekaboo + switch	0.141 ±0.000	0.134 ±0.038	0.084 ±0.021	0.052 ±0.002	0.051 ±0.004	0.027 ±0.000
	pooled Peekaboo without the honesty fix	0.227 ±0.020	0.261 ±0.024	0.253 ±0.027	0.202 ±0.013	0.321 ±0.006	0.056 ±0.005
Laya	base model	0.212	0.113	0.174	0.183	0.306	0.012
	Peekaboo	0.157 ±0.004	0.063 ±0.012	0.071 ±0.006	0.052 ±0.005	0.086 ±0.014	0.018 ±0.004
	Peekaboo + switch	0.212 ±0.000	0.084 ±0.007	0.174 ±0.000	0.052 ±0.005	0.306 ±0.000	0.015 ±0.005
	Peekaboo without the honesty fix	0.319 ±0.009	0.228 ±0.002	0.329 ±0.027	0.351 ±0.014	0.347 ±0.012	0.059 ±0.000
	Peekaboo without the KL term ($\beta = 0$)	0.178 ±0.015	0.066 ±0.015	0.065 ±0.008	0.056 ±0.013	0.088 ±0.018	0.018 ±0.002
	Peekaboo, encoder unfrozen	0.152 ±0.006	0.087 ±0.012	0.062 ±0.009	0.062 ±0.019	0.035 ±0.007	0.030 ±0.007
	Peekaboo, encoder unfrozen + switch	0.189 ±0.033	0.092 ±0.012	0.062 ±0.009	0.062 ±0.019	0.035 ±0.007	0.012 ±0.000
	Peekaboo, encoder unfrozen, no honesty fix	0.318 ±0.009	0.293 ±0.008	0.244 ±0.032	0.240 ±0.005	0.114 ±0.017	0.066 ±0.007
	pooled Peekaboo (one add-on, all suites' feedback)	0.164 ±0.007	0.096 ±0.019	0.067 ±0.015	0.052 ±0.016	0.087 ±0.006	0.016 ±0.003
	pooled Peekaboo + switch	0.212 ±0.000	0.096 ±0.019	0.174 ±0.000	0.052 ±0.016	0.306 ±0.000	0.012 ±0.000
	pooled Peekaboo without the honesty fix	0.328 ±0.033	0.271 ±0.029	0.349 ±0.054	0.328 ±0.036	0.364 ±0.019	0.056 ±0.009

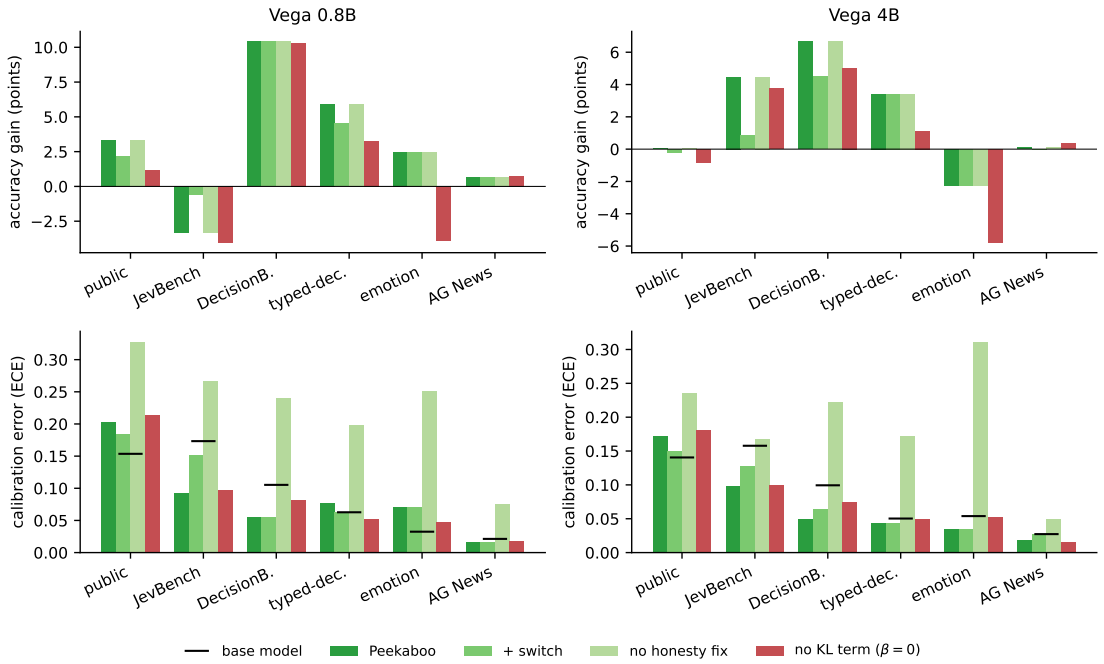


Figure 4: Ablations on Vega 0.8B (left) and 4B (right): accuracy gain (top) and ECE (bottom; the black ticks are the base model). Dropping the KL term costs accuracy on five of six benchmarks; dropping the honesty fix leaves accuracy unchanged and multiplies ECE; the switch gives up some gain for safety.

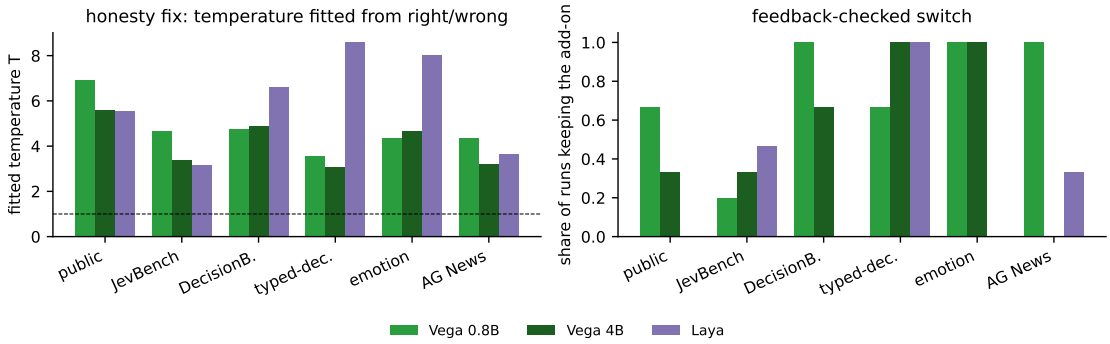


Figure 5: Left: the temperature fitted from right / wrong feedback for each model and benchmark (dashed line at $T = 1$). Right: the share of runs in which the feedback-checked switch kept the add-on.

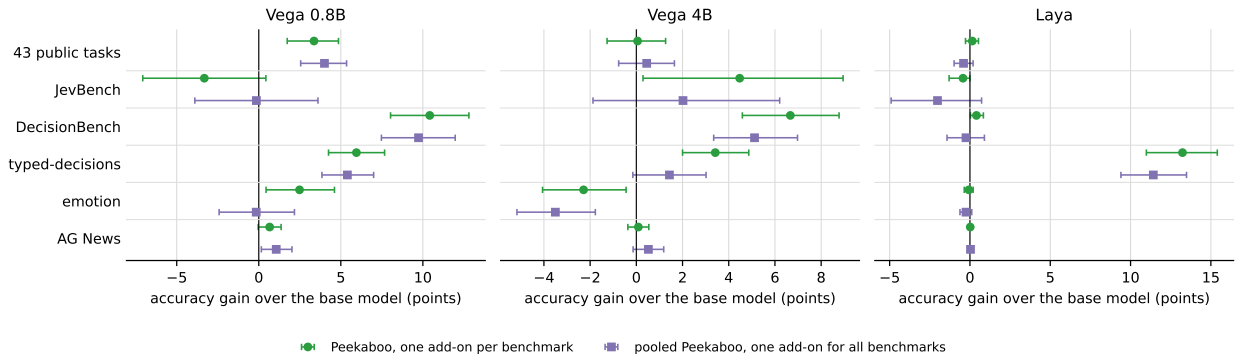


Figure 6: One add-on per benchmark (green circles) against a single pooled add-on trained on the feedback of all five benchmarks that have a training split (purple squares). Gain over the base model in points with paired 95% intervals, as in Figure 2. The pooled add-on never saw a JevBench row.

6.5 Laya, and how much of the model Peekaboo is allowed to move

Laya's own improvement is a whole-model fine-tune on labels, and on its home benchmark it is very strong: typed-decisions goes from 0.361 to 0.762, a gain of 40.2 points (the README says 0.766). Peekaboo on the frozen-encoder head gets +13.2 [+11.0, +15.4] from right / wrong alone, and our answer-key head, with full labels, gets only +18.3. So on Laya the head is the limiting factor rather than the feedback. We then let Peekaboo have the encoder too, at Laya's own encoder learning rate, with the same one-bit signal. That gives +32.3 [+29.8, +34.8] on typed-decisions, accuracy 0.684 (Table 5, Figure 7): well past the answer-key head and still short of the fine-tune, which sees $\log_2 k$ bits per row where Peekaboo sees one.

Away from typed-decisions the comparison turns around. The fine-tune loses 5.2 points on JevBench and gains little elsewhere (+0.9 on the public menu, +3.7 on DecisionBench), which is the usual price of specialising a whole model to one dataset. The head-only Peekaboo does nothing outside typed-decisions, within ±0.4. The encoder-unfrozen Peekaboo gains +3.0 [+1.7, +4.4] on the public menu (0.572 against the fine-tune's 0.551), +15.8 [+13.2, +18.7] on DecisionBench (0.559 against 0.438), and +22.0 [+19.8, +24.3] on emotion, where it reaches 0.817. That emotion figure is the highest any model or method reaches in this paper, from right / wrong alone, on the benchmark where Peekaboo on Vega looked weakest. On JevBench it loses 3.0 and on AG News 0.7. Set directly against the fine-tune the differences are +2.1 on the public menu, +2.2 on JevBench, +12.1 on DecisionBench, −7.9 on typed-decisions and +21.3 on emotion. Table 7 breaks typed-decisions down by workflow.

6.6 How much feedback, a single pass, tasks never seen

The grid grew out of an earlier DecisionBench study, which measured how the gain grows with the amount of feedback on a fixed 1,720-row test set (Figure 8). At 0.8B the gain is +3.1, +5.0, +8.4 and +11.3 points with 5, 15, 30 and 60 feedback rows per task; at 4B, +0.5, +3.0, +5.0 and +6.7. From 15 rows per task upward the feedback-only curve tracks the answer-key one, and at 4B runs a little above it. The stronger KL term ($\beta = 1$) is stable but keeps only about a third of the gain at 60 rows, which is why we settled on 0.1. Seeing each of the 2,560 rows exactly once, as a live system would, gives +6.9 at 0.8B and +3.3 at 4B. Feedback on 33 tasks improves 10 tasks that contributed none by +4.2 and +4.6 points. The no-engine control, the same feedback and the same KL term on a bilinear scorer over the same frozen features, reaches an accuracy of 0.26 at either size. Whatever one bit per row is doing, it is doing it by nudging Vega's zero-shot energies

Table 7: typed-decisions accuracy per workflow (500 test decisions each).

model	method	agent traces	customer service	invoices	security
Vega 0.8B	base	0.698	0.662	0.652	0.650
	TTT head	0.690	0.716	0.800	0.678
	answer-key adapter	0.703	0.743	0.805	0.687
	Peekaboo	0.705	0.731	0.761	0.703
Vega 4B	base	0.688	0.800	0.804	0.626
	TTT head	0.700	0.800	0.808	0.706
	answer-key adapter	0.707	0.813	0.815	0.763
	Peekaboo	0.695	0.802	0.801	0.757
Laya	base	0.388	0.380	0.360	0.316
	laya-typed-decisions	0.732	0.766	0.810	0.742
	answer-key head	0.507	0.501	0.619	0.551
	Peekaboo, head only	0.437	0.473	0.535	0.528
	Peekaboo, encoder unfrozen	0.655	0.727	0.684	0.671

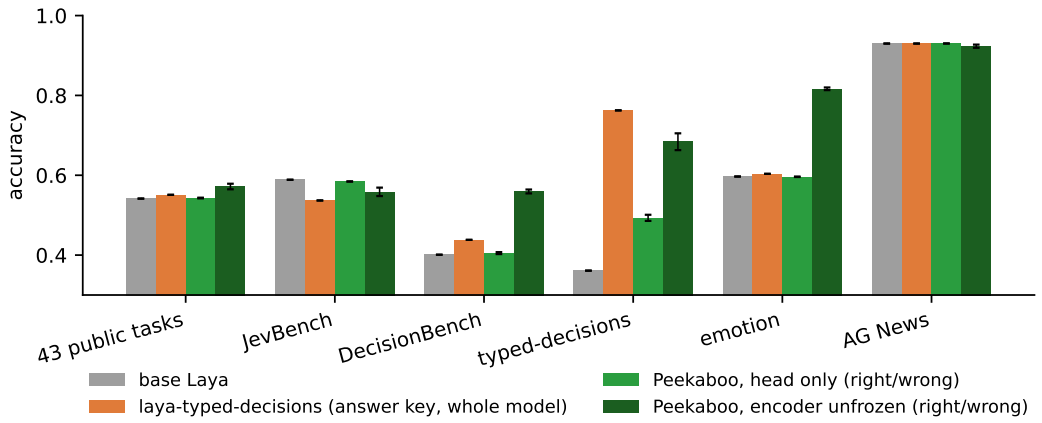


Figure 7: Laya accuracy on the six benchmarks: base, Laya's released answer-key fine-tune of the whole model, Peekaboo on the decision head alone, and Peekaboo with the encoder unfrozen.

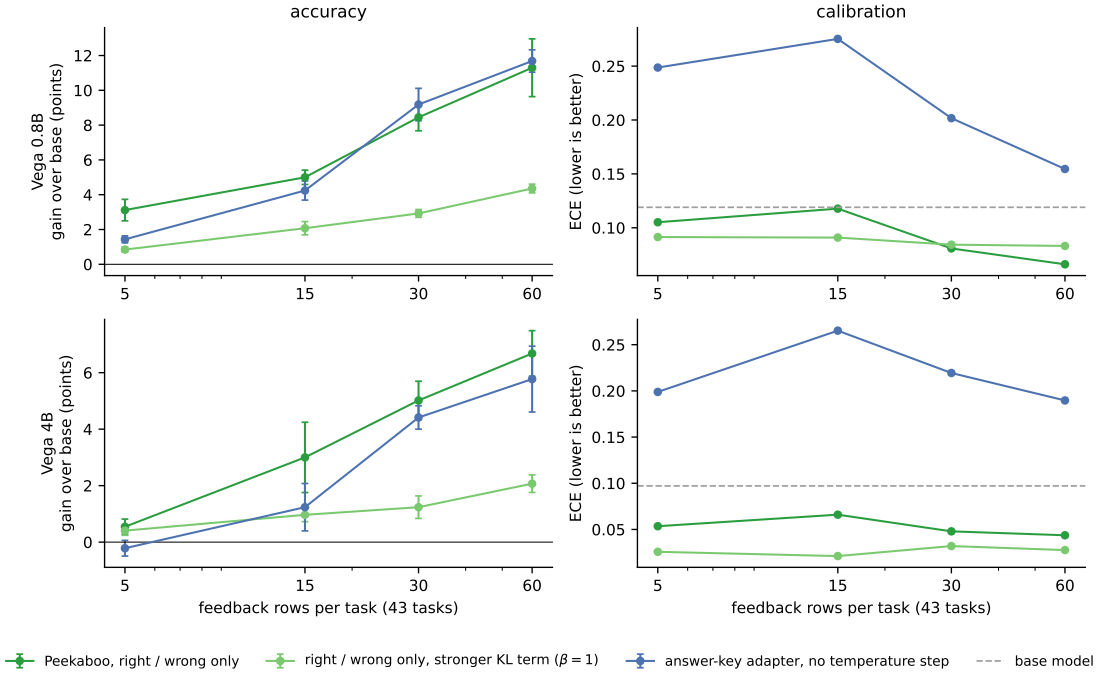


Figure 8: How the gain grows with feedback. DecisionBench, fixed 1,720-row test set: accuracy gain over the base model (left) and ECE (right) against the number of feedback rows per task, for Vega 0.8B (top) and 4B (bottom). Green: Peekaboo. Light green: the same with a stronger KL term ($\beta = 1$). Blue: the adapter trained on the answer key, which in this earlier study had no temperature step, hence its ECE. Dashed grey: the base model's ECE. Error bars are s.d. over three seeds.

rather than by learning the task from scratch.

6.7 Reward during training

Figure 9 shows the mean reward of the sampled answers over the eight passes. On Vega it climbs on every benchmark with a training split (DecisionBench at 0.8B: 0.50 to 0.73). On Laya's frozen-encoder head it

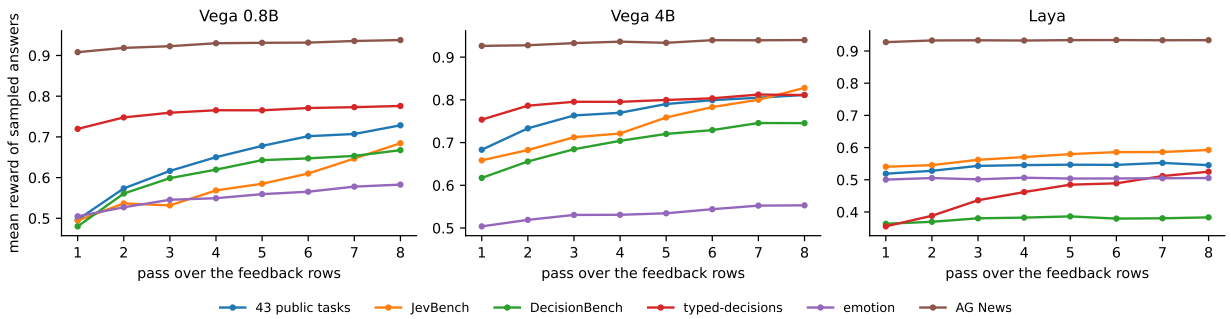


Figure 9: Mean reward of the sampled answers in each of the eight passes over the feedback rows, averaged over seeds.

moves on typed-decisions and nowhere else, which agrees with the accuracy results. Because a sampled answer is right less often than the argmax, this curve also bounds what exploration would cost a live system: on the first pass the policy is right about as often as the base model's own samples, and more often from the second pass on.

7 Discussion

The case for Peekaboo is a model that is already deployed, collects right / wrong events as a by-product of being used, and has to stay competent at everything it already does. In that setting, with the same capacity, right / wrong recovers most of what the answer key would give, the KL term keeps the add-on from wandering, the switch catches regressions, and the honesty fix keeps the probabilities honest enough to route on. The gains were largest where the base model already leaned towards the right option and had it weighted wrong (DecisionBench's online-safety, home-automation and web domains in the earlier study) and smallest, or negative, where the task needs reasoning the base model simply lacks (DecisionBench's logic domain; the temporal-numeric family of JevBench at 0.8B). An add-on of a few hundred thousand parameters is not going to teach a frozen model arithmetic.

The case against it is a fixed task with thousands of fully labelled rows and permission to retrain the whole model. Laya's typed-decisions fine-tune at +40 is the clearest example, and VegaML's TTT head on emotion is another. The answer key carries $\log_2 k$ bits per row to Peekaboo's one, and when the task is a clean single-label classification with many rows and the add-on is small, that gap shows up in the accuracy. What the Laya encoder-unfrozen runs taught us is that the gap is partly about capacity rather than about the signal. Given the encoder, one bit per row took Laya's emotion accuracy from 0.597 to 0.817, where Laya's own fine-tune and VegaML's TTT top out at 0.604 and 0.651. We do not have the matching experiment for Vega, whose backbone is frozen by design, and we would like to.

The TTT losses on the public menu and DecisionBench have a mundane explanation. A TTT head is a per-task classifier over a fixed label set. On DecisionBench many rows carry their own option list (a rating scale worded differently per row, a list of candidate records to pick from), so `fit()` kept heads for 20 of the 43 tasks, covering 394 of the 1,075 rows, and fell back to the engine for the rest. Peekaboo changes the engine, and the engine reads the option text, so it reaches every row. On the public menu, 50 labelled rows per task turn out to be enough for a logistic head to be confidently wrong on tasks where Vega's zero-shot answer was right.

8 Limitations

- **Simulated feedback.** Our rewards come from gold answers, so they are never wrong, never missing and never skewed towards complaints. Real feedback is all of those things. The method only needs a 0/1 signal, and the code has the knobs for studying the rest (flip probability, report probability conditional on outcome, logged off-policy feedback with clipped importance weights), but we have not turned them, and the results here should be read as the clean-signal ceiling.
- **Exploration costs something.** During learning the model samples instead of taking the argmax. Figure 9 bounds that cost but we did not design a conservative exploration policy, for instance one that only explores inside Vega's conformal set.
- **Hyper-parameters.** $\beta = 0.1$ was picked in a pilot on part of DecisionBench and never retuned; rank 8, 8 passes and the 10^{-3} learning rate were fixed before we had any results. We also did not tune TTT (VegaML defaults) or the answer-key add-on separately from Peekaboo. The same settings are used on all six benchmarks.
- **One temperature per benchmark.** On the 43-task public menu this is too coarse, and it is the one place Peekaboo is worse calibrated than the base model. A per-task or per-option-count temperature, fitted the same way, is the obvious thing to try.
- **Small benchmarks.** JevBench has 231 rows and 42-row calibration folds. Its intervals are wide and the switch is unreliable there.
- **Vega on typed-decisions.** Vega's README reports typed-decisions on six unpublished workflows. We could only use Laya's public release, so Vega's own typed-decisions figure has no counterpart in our tables.

9 Conclusion

The natural expectation is that the answer key wins comfortably and the only question is by how much. On Vega it mostly does not win. A rank-8 adapter trained on nothing but whether a sampled answer was right comes within a point or two of the same adapter trained on labels, and beats VegaML's own labelled-data method on four of the six benchmarks the authors chose, at both model sizes. The two things we had added as precautions, the KL term and the feedback-fitted temperature, turned out to be where the method lives: without the first the gains shrink or reverse, and without the second the probabilities are off by a factor the temperatures in Figure 5 make plain. Laya complicated the picture in a useful way. There the bit was not the bottleneck; the head was, and once the encoder was in play the same bit carried Laya past its own fine-tune on four of five benchmarks away from home while recovering most of the fine-tune's gain at home. The open question this leaves, which we could not test because Vega's backbone is frozen, is whether the same holds for Vega if more of the model were reachable, and whether feedback that is noisy, sparse and selective, as real feedback is, leaves any of this standing.

Acknowledgements. We thank the authors of VegaML and Laya for releasing their models, benchmark code and training recipes. Without the harness and the recipes a like-for-like comparison would not have been possible.

References

[1] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, 2017.

[2] Convai Innovations. Laya: fast, non-autoregressive System 1 decision engine with calibrated probabilities. https://huggingface.co/convaiinnovations/laya, 2026. Python package `laya` 0.3.23; checkpoints `convaiinnovations/laya` and `convaiinnovations/laya-typed-decisions`.

[3] Miroslav Dudík, John Langford, and Lihong Li. Doubly robust policy evaluation and learning. In *Proceedings of ICML*, 2011.

[4] Bradley Efron. Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[5] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. KTO: model alignment as prospect theoretic optimization. In *Proceedings of ICML*, 2024.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of ICML*, 2017.

[7] Hanno Labs. DecisionBench. https://huggingface.co/datasets/Hanno-Labs/decision-bench, 2026.

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of ICLR*, 2022.

[9] Natasha Jaques, Asma Ghandeharioun, Judy Hanwen Shen, Craig Ferguson, Agata Lapedriza, Noah Jones, Shixiang Gu, and Rosalind Picard. Way off-policy batch deep reinforcement learning of implicit human preferences in dialog. In *arXiv preprint arXiv:1907.00456*, 2019.

[10] Julia Kreutzer, Joshua Uyheng, and Stefan Riezler. Reliability and learnability of human bandit feedback for sequence-to-sequence reinforcement learning. In *Proceedings of ACL*, 2018.

[11] Harold W. Kuhn. A simplified two-person poker. In *Contributions to the Theory of Games*, volume 1, pages 97–103, 1950.

[12] John Langford and Tong Zhang. The epoch-greedy algorithm for multi-armed bandits with side information. In *Advances in Neural Information Processing Systems*, 2008.

[13] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web*, 2010.

[14] LocalLLaMA. typed-decisions. https://huggingface.co/datasets/LocalLLaMA/typed-decisions, 2026.

[15] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning into quantiles. In *Proceedings of AAAI*, 2015.

[16] NandhaKishorM. VegaML: typed decisions from a frozen language model and a small physics engine. https://github.com/NandhaKishorM/vegaml, 2026. Python package `vegaml` 0.8.0; models `nandakishorm/vega-08b-public-intents` and the 4B release.

[17] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.

[18] John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, 1999.

[19] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[20] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.

[21] Elvis Saravia, Hsien-Chi Toby Liu, Yen-Hao Huang, Junlin Wu, and Yi-Shin Chen. CARER: Contextualized affect representations for emotion recognition. In *Proceedings of EMNLP*, 2018.

[22] Artem Sokolov, Julia Kreutzer, Christopher Lo, and Stefan Riezler. Learning structured predictors from bandit feedback for interactive NLP. In *Proceedings of ACL*, 2016.

[23] Florian Standhartinger. JevBench: public tiers. https://huggingface.co/datasets/fstandhartinger/jevbench, 2026.

[24] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of ICML*, 2020.

[25] Adith Swaminathan and Thorsten Joachims. Counterfactual risk minimization: learning from logged bandit feedback. In *Proceedings of ICML*, 2015.

[26] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, et al. Smarter, better, faster, longer: a modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.

[27] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

[28] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*, 2015.

A Reproducing the results

`experiments/peekaboo_remaining/run.py` runs the whole grid. It reads every benchmark once through every model and caches the backbone features and the official records, replays each base model against its official prediction call, then runs every method and seed and writes `output/TABLES.md`, from which `paper/make_figures.py` produces every table and figure here. Finished steps are skipped, so the run can be resumed; failed steps are retried and listed in `output/STATUS.md`. `experiments/vega_policy/reproduce.py` runs the DecisionBench study of Section 6.6: feedback amount, the single pass, unseen tasks, latency, the no-engine control, and the Kuhn-poker [11] experiment that this work started from. A second person reproduced the 0.8B DecisionBench gain (+10.4 against our +10.2) and the released 4B accuracy (0.635 against the README's 0.633) on a different machine before the grid was run.

Checks. Replaying each frozen model on its cached inputs matches the official prediction call to 0 for Vega and to 5×10^{-5} for Laya. The Laya replay is one question per forward pass, as Laya's own call runs it; padded batches move its probabilities by up to 0.02 under autocast. Base Vega 0.8B and 4B reproduce the README's public-menu accuracy (0.556 and 0.715 against 0.556 and 0.717); base Laya and `laya-typed-decisions` reproduce the README's typed-decisions accuracy (0.361 and 0.762 against 0.362 and 0.766).

B Per-task results on the public menu

Table 8 lists all 43 public tasks. Peekaboo's largest gains over base Vega 0.8B are on prompt_injections (0.75 → 0.87), superglue_cb (0.67 → 0.77) and tweet_sentiment (0.44 → 0.54). Its losses fall on tasks where the TTT head loses as well: unfair_tos (0.74 → 0.53, TTT 0.63) and arc_easy (0.60 → 0.57, TTT 0.35). On Laya the head-only add-on barely moves any task. The encoder-unfrozen variant lifts the safety tasks (aegis_prompt_safety 0.37 → 0.69, prompt_injections 0.65 → 0.78) and the rating and topic tasks (yelp_stars 0.28 → 0.50, trec_coarse 0.53 → 0.71), and gives back some ground on intent (banking77 0.40 → 0.33).

Table 8: Accuracy on each of the 43 public tasks (up to 60 test rows each; mean over seeds). "key" is the answer-key adapter / head, "Pk" is Peekaboo, "fine-tune" is laya-typed-decisions, "Pk enc" is Peekaboo with the encoder unfrozen. Bold: the best method for that model on that task.

task	Vega 0.8B				Vega 4B				Laya			
	base	TTT	key	Pk	base	TTT	key	Pk	base	fine-tune	Pk head	Pk enc
aegis_prompt_safety	0.50	0.58	**0.65**	0.64	0.60	0.60	**0.78**	0.73	0.37	0.43	0.44	**0.69**
ag_news	0.97	0.92	0.97	**0.98**	0.95	0.95	**0.96**	0.94	**0.98**	**0.98**	**0.98**	**0.98**
anli_r3	0.27	**0.45**	0.40	0.38	0.35	**0.50**	0.46	0.41	0.43	0.40	0.42	**0.45**
arc_challenge	0.47	0.35	0.46	**0.48**	0.78	0.77	0.76	**0.81**	0.28	0.37	0.28	**0.38**
arc_easy	**0.60**	0.35	0.52	0.57	0.92	0.85	0.93	**0.94**	0.52	**0.58**	0.52	0.57
banking77	0.68	0.68	**0.71**	0.63	0.73	0.73	0.72	**0.75**	**0.42**	0.40	0.41	0.33
boolq	0.77	0.62	0.67	**0.81**	0.82	0.85	0.82	**0.87**	0.87	0.85	**0.88**	**0.88**
casehold	0.35	0.35	**0.51**	0.45	0.55	0.42	**0.68**	0.65	0.37	0.38	0.37	**0.41**
climate_fever	0.38	0.40	0.46	**0.49**	0.45	0.37	0.46	**0.50**	0.33	**0.35**	0.33	0.29
clinc150_oos	**0.67**	**0.67**	0.56	0.62	**0.90**	**0.90**	0.72	0.80	0.00	**0.32**	0.00	0.00
commonsenseqa	0.35	0.15	**0.42**	0.36	0.53	0.48	0.59	**0.61**	**0.45**	0.43	**0.45**	0.43
cruxeval_output	0.65	0.48	**0.88**	0.78	0.85	0.92	0.92	**0.95**	**0.78**	0.73	0.78	0.78
dbpedia14	0.60	**0.85**	0.77	0.71	0.90	**0.97**	0.91	0.92	0.88	0.83	0.88	**0.89**
emotion	**0.55**	0.47	0.36	0.52	**0.65**	0.45	0.40	0.57	**0.57**	**0.57**	**0.57**	0.50
enron_spam	**1.00**	0.92	0.99	0.99	**1.00**	0.98	**1.00**	**1.00**	0.92	0.90	0.94	**0.94**
glue_rte	**0.63**	0.48	0.59	0.60	**0.88**	0.85	0.84	0.85	**0.83**	0.82	**0.83**	0.77
hellaswag	0.25	0.25	**0.26**	**0.26**	**0.65**	0.52	0.49	0.56	0.37	0.37	0.35	**0.42**
jailbreak_cls	**0.98**	**0.98**	0.98	**0.98**	**1.00**	0.98	0.99	0.99	**0.98**	**0.98**	**0.98**	**0.98**
ledgar	0.32	0.32	**0.62**	0.40	**0.65**	**0.65**	0.64	0.63	**0.25**	0.20	**0.25**	0.14
massive_intent_en	0.38	0.38	**0.49**	0.42	0.60	0.60	**0.63**	0.59	0.38	**0.43**	0.38	0.40
medmcqa	0.18	**0.28**	0.24	0.27	0.45	0.50	**0.53**	0.47	**0.35**	**0.35**	**0.35**	**0.35**
medqa_usmle	**0.29**	**0.29**	0.27	0.27	0.51	0.36	0.49	**0.53**	0.22	0.27	0.22	**0.28**
mmlu	**0.37**	0.22	0.29	0.32	**0.72**	0.42	0.61	0.65	0.27	**0.35**	0.27	0.32
mmlu_pro	0.20	0.13	0.14	**0.22**	0.38	0.18	**0.43**	0.41	0.18	0.20	0.19	**0.21**
openbookqa	**0.58**	0.22	0.52	**0.58**	**0.83**	0.63	0.73	0.77	0.42	**0.50**	0.42	0.49
prompt_injections	0.75	**0.96**	0.87	0.87	0.85	**0.96**	0.81	0.88	0.65	0.60	0.72	**0.78**
pubmedqa	0.13	**0.57**	0.48	0.54	0.40	**0.63**	0.61	0.57	0.53	0.55	0.53	**0.57**
race_middle	**0.60**	0.37	0.52	**0.60**	0.80	**0.88**	0.82	0.82	**0.58**	0.52	0.58	0.55
sciq	**0.73**	**0.73**	0.58	0.72	0.93	0.87	0.93	**0.94**	0.52	0.53	0.52	**0.59**
superglue_cb	0.67	0.67	0.53	**0.77**	0.82	**0.89**	0.78	0.80	0.87	0.87	**0.88**	0.81
toxic_chat_jailbreak	**0.98**	**0.98**	0.96	0.93	0.93	0.45	**0.95**	0.92	0.83	0.90	0.81	**0.92**
toxic_chat_toxicity	**0.95**	0.93	0.91	0.91	**1.00**	0.98	0.98	0.98	0.97	0.97	**0.98**	0.90
trec_coarse	0.53	**0.72**	0.71	0.43	0.77	**0.88**	0.82	0.75	0.53	0.53	0.52	**0.71**
truthfulqa_mc1	0.36	0.36	**0.81**	0.75	0.62	0.62	**0.75**	**0.75**	0.16	0.24	0.16	**0.43**
tweet_hate	**0.65**	0.63	0.58	0.62	**0.68**	0.60	0.54	0.50	**0.73**	0.72	0.72	0.62
tweet_irony	**0.78**	0.62	0.71	0.74	**0.78**	0.67	0.68	0.72	**0.78**	0.74	0.75	0.68
tweet_offensive	0.78	0.80	0.80	**0.81**	0.78	0.81	0.80	**0.84**	0.69	0.71	0.67	**0.79**
tweet_sentiment	0.44	0.41	0.45	**0.54**	**0.58**	**0.58**	0.50	0.49	0.56	0.49	**0.58**	0.54
tweet_stance_abortion	0.36	0.47	**0.48**	0.44	**0.53**	0.49	0.47	0.42	0.33	0.31	0.33	**0.45**
unfair_tos	**0.74**	0.63	0.56	0.53	**0.88**	0.74	0.78	0.76	**0.88**	0.84	0.83	0.70
vitaminc	**0.47**	0.28	0.42	0.39	**0.62**	0.57	0.53	0.61	0.43	0.32	0.46	**0.61**
winogrande	0.50	0.50	0.48	**0.54**	0.57	0.57	**0.62**	0.61	0.53	0.48	0.53	**0.57**
yelp_stars	0.52	0.52	**0.54**	0.50	**0.57**	0.48	0.47	0.52	0.28	0.37	0.28	**0.50**

C Hyper-parameters

add-on, Vega	gated LoRA, rank 8, on the engine's input layers (`vegaml.attach_adapter`); 409k / 857k parameters
add-on, Laya	trainable copy of type embedding, 2-layer decision head and scorer; encoder frozen, or a trained copy ("encoder unfrozen")
optimiser	Adam; lr 10^{-3} (Vega adapter), 10^{-4} (Laya head), 2.5×10^{-5} (Laya encoder); grad-norm clip 1.0
schedule	8 passes, batches of 32 (Laya encoder: micro-batches of 8, one update per 32 rows)
reward	$r = \mathbf{1}[\text{sampled answer} = \text{gold}]$; baseline $b \leftarrow 0.9\, b + 0.1\, \bar{r}$, $b_0 = 0.5$
KL term	$\beta = 0.1$ (ablation: 0)
honesty fix	T on a 61-point log grid in $[0.25, 32]$, maximising the Bernoulli likelihood of right / wrong on the calibration rows
splits	feedback rows 80 / 20 train / calibration per task by a fixed hash; JevBench 5-fold by paraphrase group
seeds	0, 1, 2